6/11/2021
Thursday

MODULE - 4

Graph Data Structure

> **Graph**

A graph is a collection of vertices 'V' and edges 'E'. Many problems are formulated in terms of objects and connections between them.

Eg:- Airline routes, electrical circuits

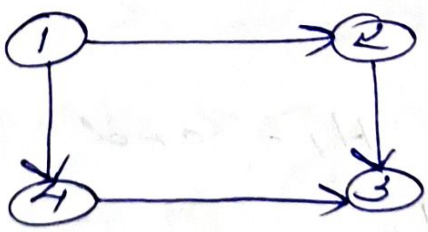
> 2 types of graphs :-

(i) Directed graphs
(ii) Undirected graphs

> **Directed graph (Digraph)**

It is an ordered pair of vertices and a direction is associated with each edge.

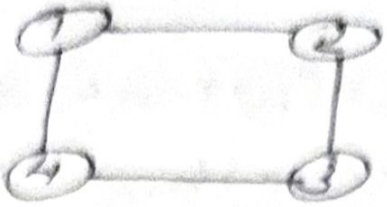
Eg:-



> **Undirected graphs**

A graph which is unordered pair of vertices and there is no direction associated with edge is called undirected graph.

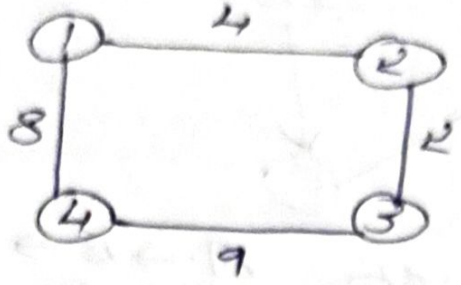
Eg :-

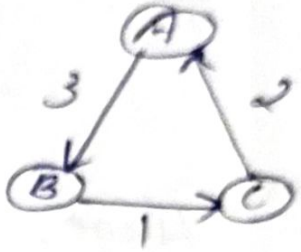


> Weighted graph

A graph in which each edge is associated with a weight.

Eg :-





> Path

A simple path is a path with no vertex repeated. Es :- A→B→C

· A simple cycle is a simple path except the first and last vertex is repeated. } cycle vs Path

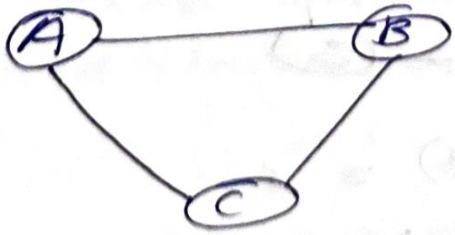
Eg :- A→B→C→A

· Length of a path (length = No: of edges)

Total number of edges included in a path.

> Length of path = n - 1

Es :-



path A→B→C

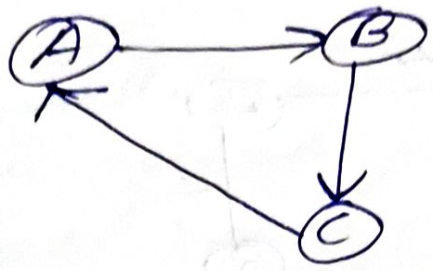
length = 2

= 3 - 1

> Cycle

In a digraph, a path $v_1, v_2, v_3, \ldots, v_{n-1}, v_n$ is called a cycle if it atleast two vertices and first and last vertices are same.

Eg:-



Here, the path $A \to B \to C \to A$ is a cycle.

> Degree :- Maximum No: of edge incident on a graph.

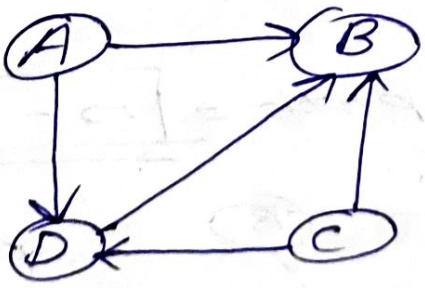
· Indegree

It is the number of edge coming to a vertex.
(entering)

· Outdegree

It is the number of edge going out of a vertex.
(leaving)

Eg:-



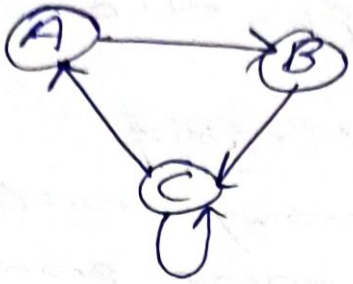
Indegree(B) = 3

Outdegree(C) = 2

> Cyclic Graph

A graph has one or more cycles is called a cyclic graph.

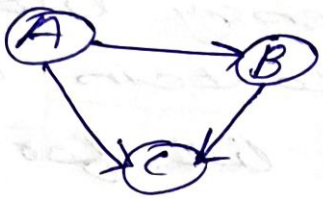
Eg:-



> Acyclic graph

A graph with no cycle is called an acyclic graph.

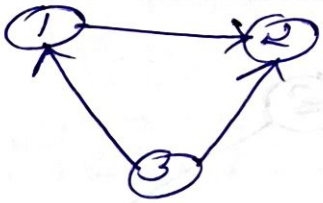
Es:-



> DAG (Directed Acyclic Graph)

A graph with no cycle but directions is called DAG.

Es:-



> Source

A vertex which has no incoming edges, but have outgoing edges.

> Sink

A vertex that have no outgoing but only incoming edges.

Pg :-

> Representation of Graph

1. Adjacency Matrix
2. Adjacency List

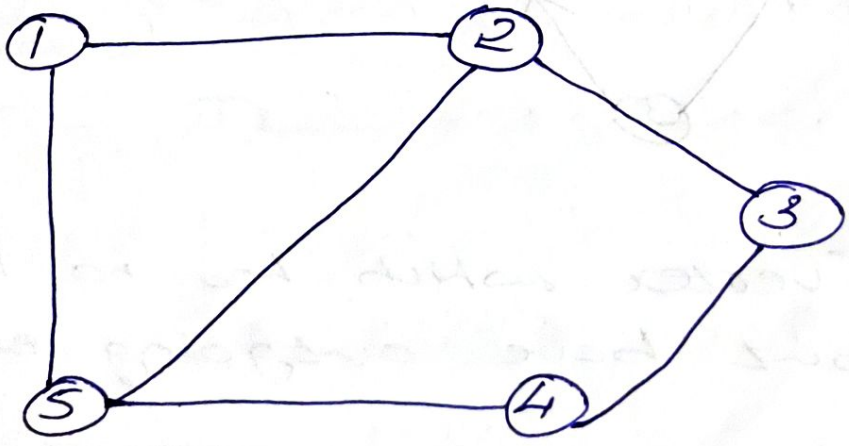
> Adjacency Matrix

An adjacency matrix is a matrix with rows and columns for each vertex. Matrix entry is 1 if there is an edge, otherwise 0.

> Adjacency List

An adjacency list, it is an array which contain a list for each vertex. The list for a given vertex contain all other vertices that are connected to the first vertex by a single edge.

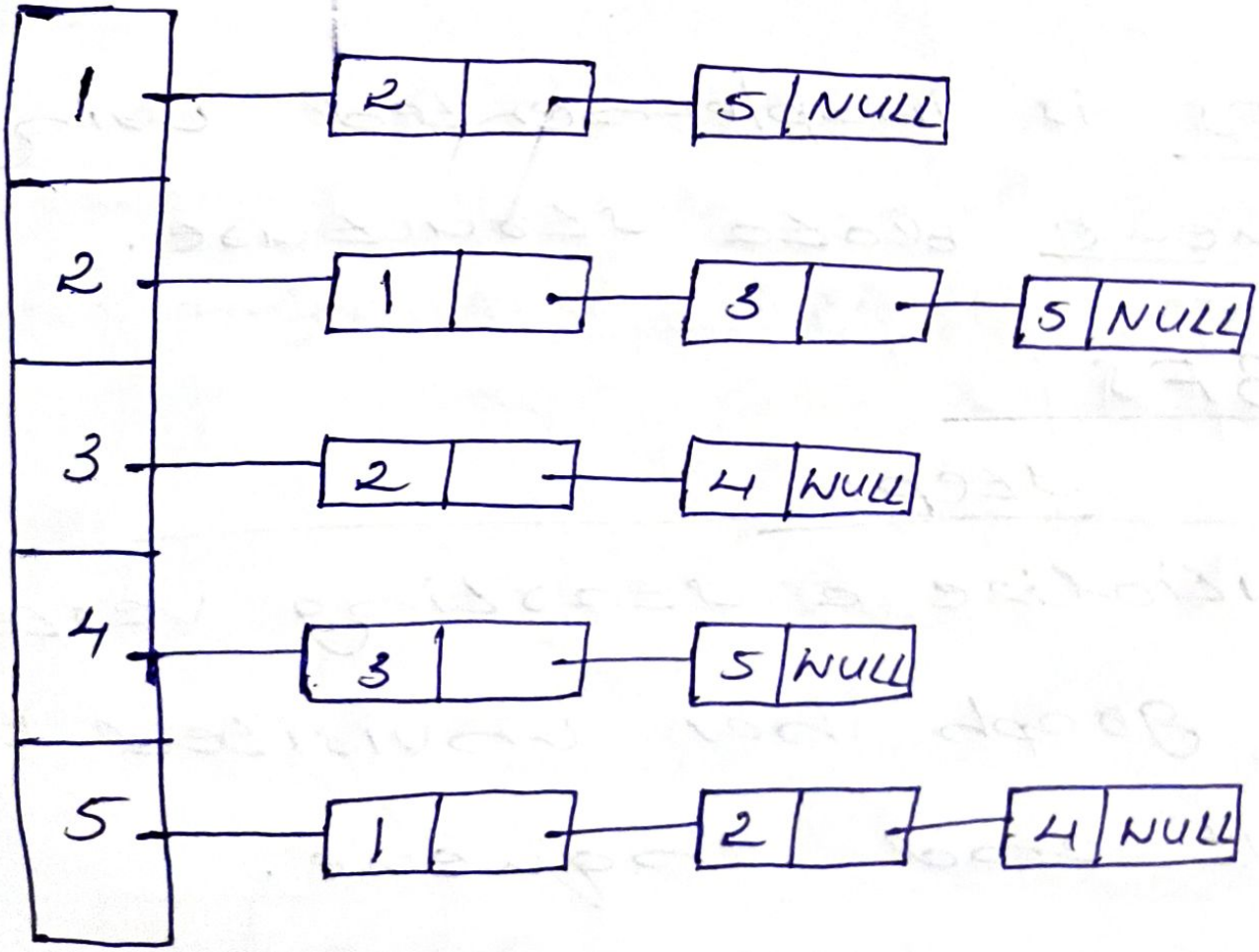
Example



Adjacency matrix

	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	0	1
3	0	1	0	1	0
4	0	0	1	0	1
5	1	1	0	1	0

Adjacency List



10/11/2025 Monday

Breadth First Search (BFS)

ALGORITHM

BFS(G, s)

for each vertex $u \in V[G] - \{s\}$

> Graph Traversal Algorithms:

Two searching algorithms:

1. BFS (Breadth First Search)
2. DFS (Depth First Search)

> • BFS is implemented using the Queue data structure.

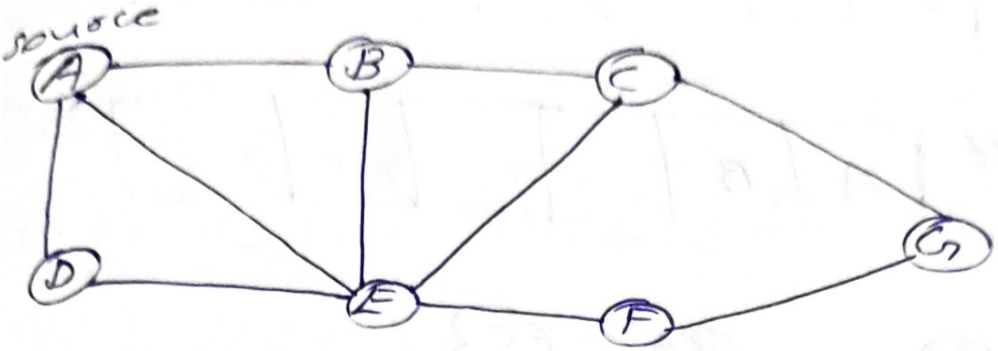
> BFS

Steps

1. Initialize a starting vertex.
2. If graph has unvisited vertex, visit and enqueue.
3. While queue is not empty, dequeue vertex and visit its adjacent unvisited node and enqueue them.

Example

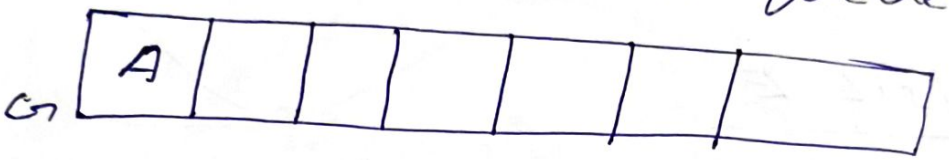
Consider the following undirected graph :-



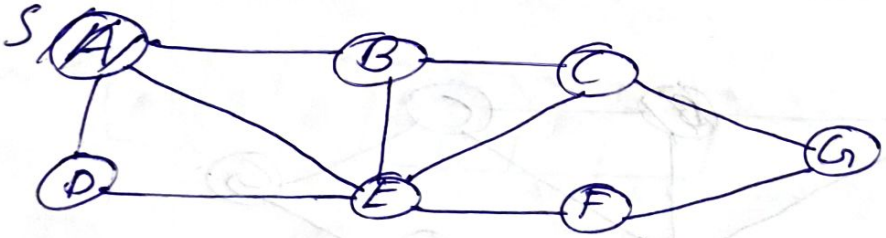
① Visit A

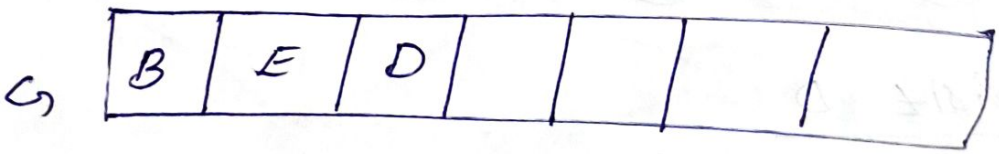
Ans:- Let source = A.

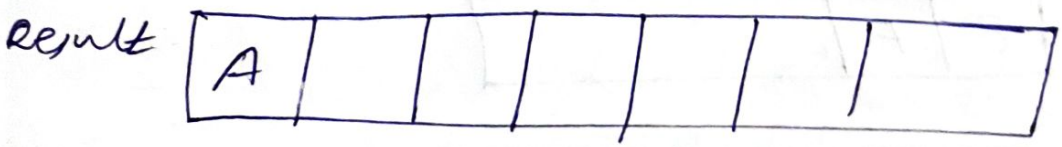
Insert A into the queue, G.



Dequeue A, add it to the resulting queue and enqueue the adjacent vertices of A to the queue G.



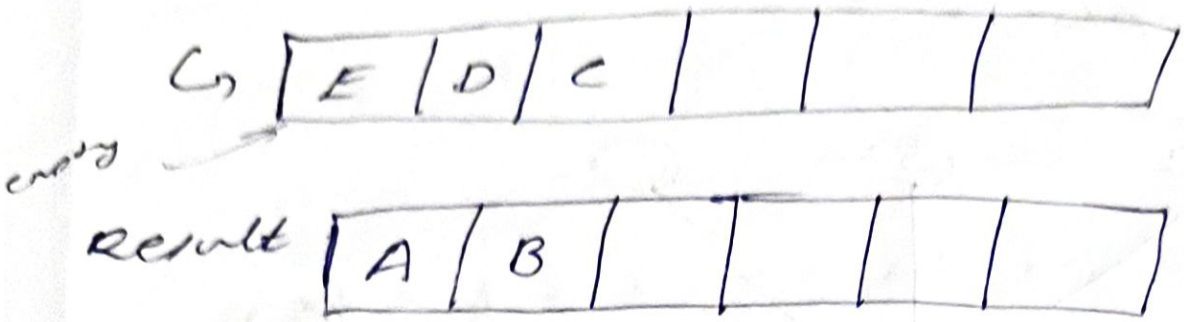


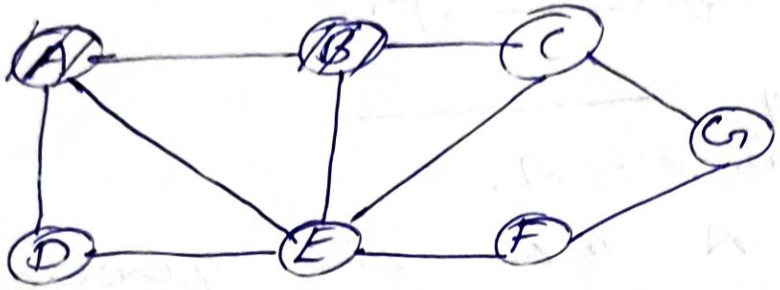


② Visit B

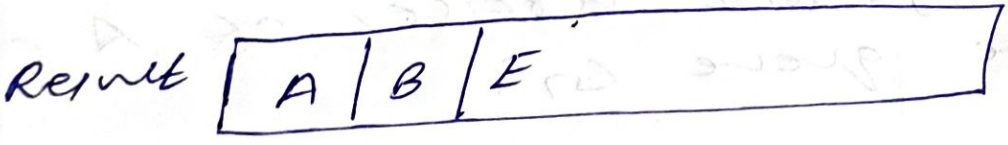
Deque B from G and add it to the 'Result' queue. Enqueue the

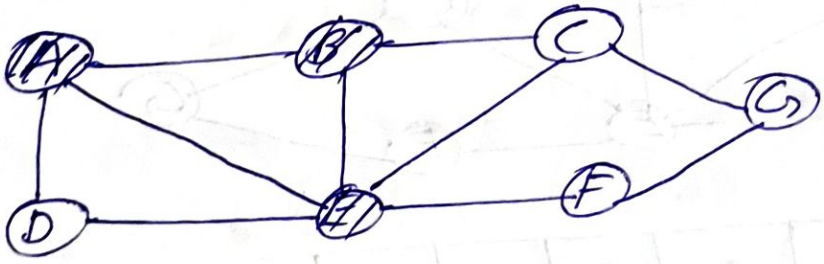
adjacent vertices of B into G and queue.



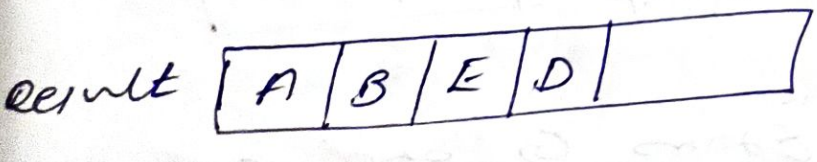


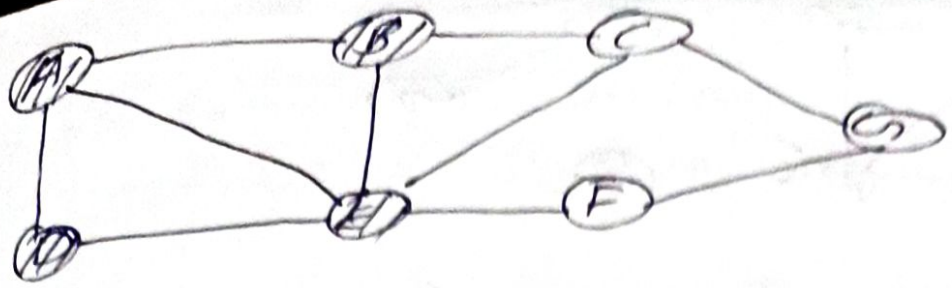
③ Visit E



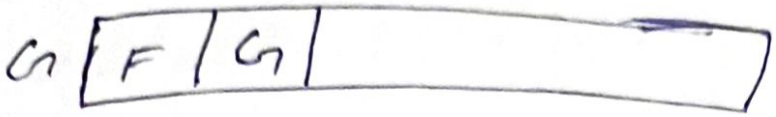


④ Visit D

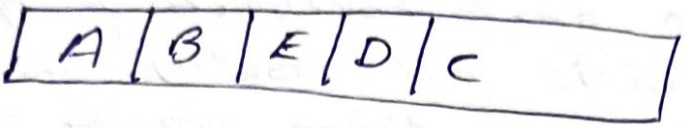


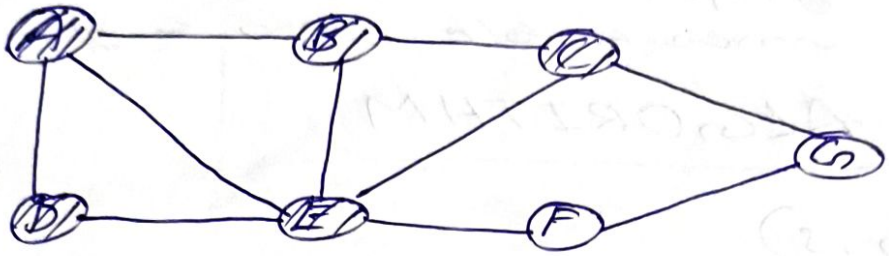


⑤ Visit C



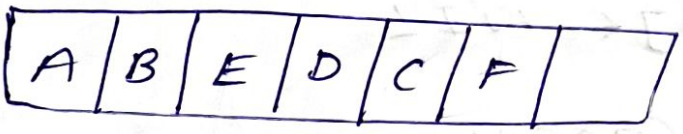
Result

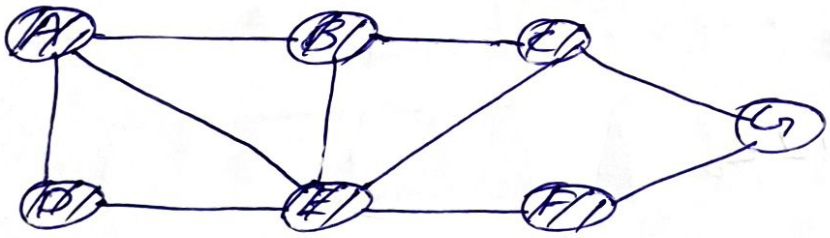




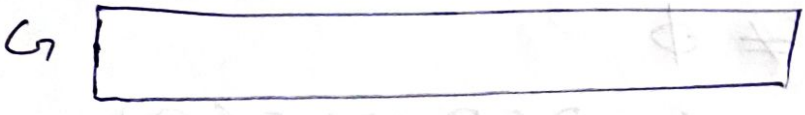
⑥ Visit F

Result

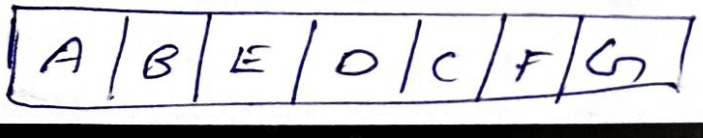


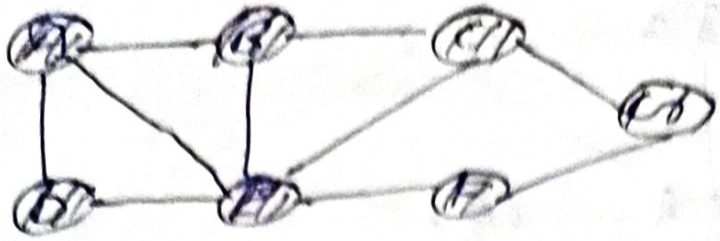


⑦ Visit G



Result





∴ BFS Traversal :-

A → B → E → D → C → F → G

Note :- For an undirected graph, BFS will visit all nodes. While in directed graph, there may be nodes that are unreachable from a given node.

ALGORITHM

BFS(G, s)

```
for each vertex u ∈ V[G] - {s}
    do color[u] ← white
       d[u] ← ∞
parent ← π[u] ← NIL
color[s] ← Gray
d[s] ← 0
π[s] ← NIL
Q ← ϕ
ENQUEUE(Q, s)
while Q ≠ ϕ
    do u ← DEQUEUE(Q)
```

For each $v \in Adj[u]$

do if color[v] ← Gray

$d[v] \leftarrow d[u] + 1$

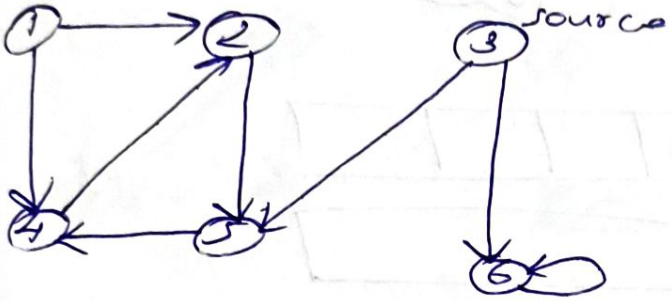
$\pi[v] \leftarrow u$

ENQUEUE(Q, v)

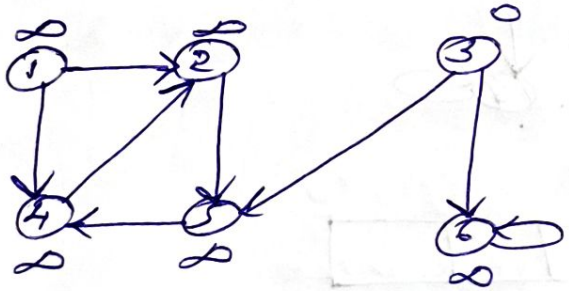
color[u] ← BLACK

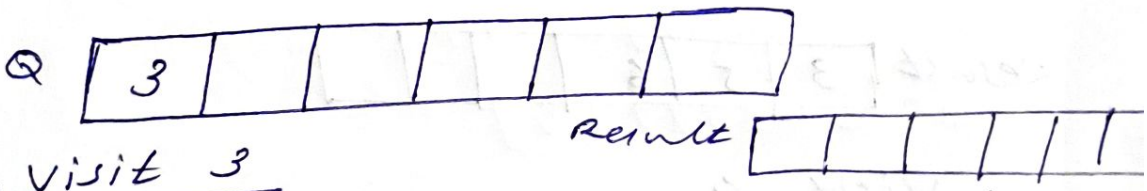
Next,

Example

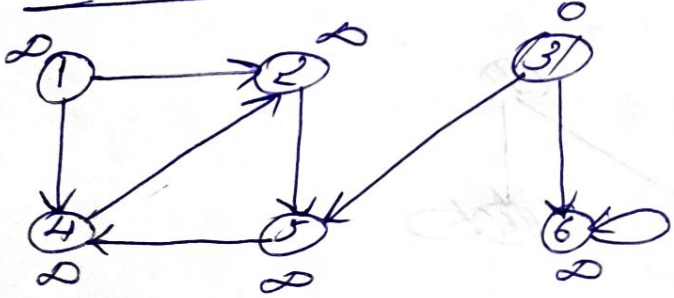


Ans: Initially, set all the distances to ∞ and the distance of source to 0.





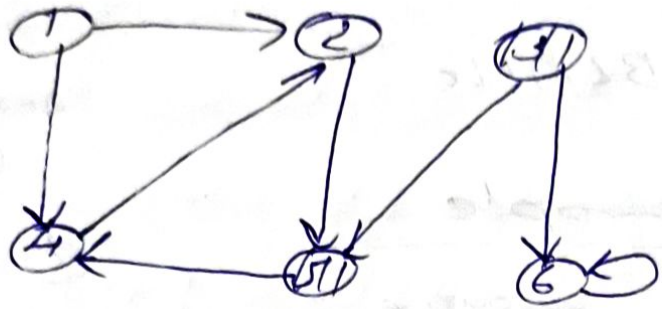
. Visit 3



Q | 5 | 6 | | | | |

Result | 3 | | | | | |

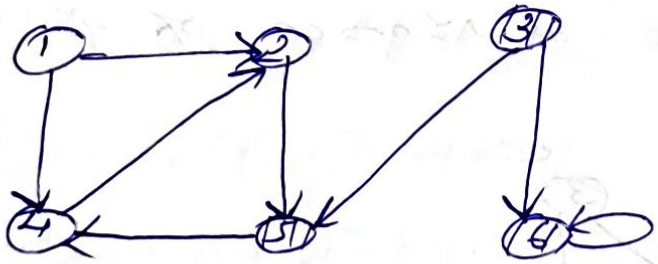
. Visit 5



Q | 6 | 4 | | | | |

Result | 3 | 5 | | | | |

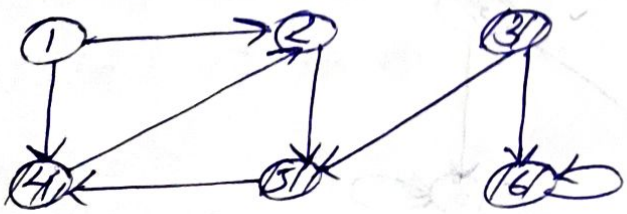
. Visit 6



Q | | 4 | | | | |

Result | 3 | 5 | 6 | | | |

. Visit 4



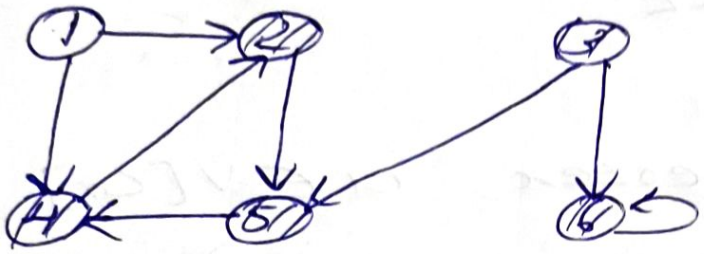
Q | | | | 2 | | |

result | 3 | 5 | 6 | 4 | | |

. visit 2

Q | | | | | | |

result | 3 | 5 | 6 | 4 | 2 | |



. Here, 1 is not visited since it is unreachable from 3.

∴ BFS traversal is:-

3 → 5 → 6 → 4 → 2

11/11/2025 Tuesday.

Depth First Search (D

. It is implemented using stack data structure.

ALGORITHM

. DFS(G)

① For each vertex $u \in V[G]$

② Do color[u] ↔ white

③ $\pi[u] = NIL$

④ time ← 0

⑤ For each vertex $u \in V[G]$

⑥ Do if (color[u] ← white)

⑦ Then DFS_VISIT[u]

. DFS_VISIT[u]

① color[u] ← grey

② time ← time + 1

③ For each $v \in adj[u]$

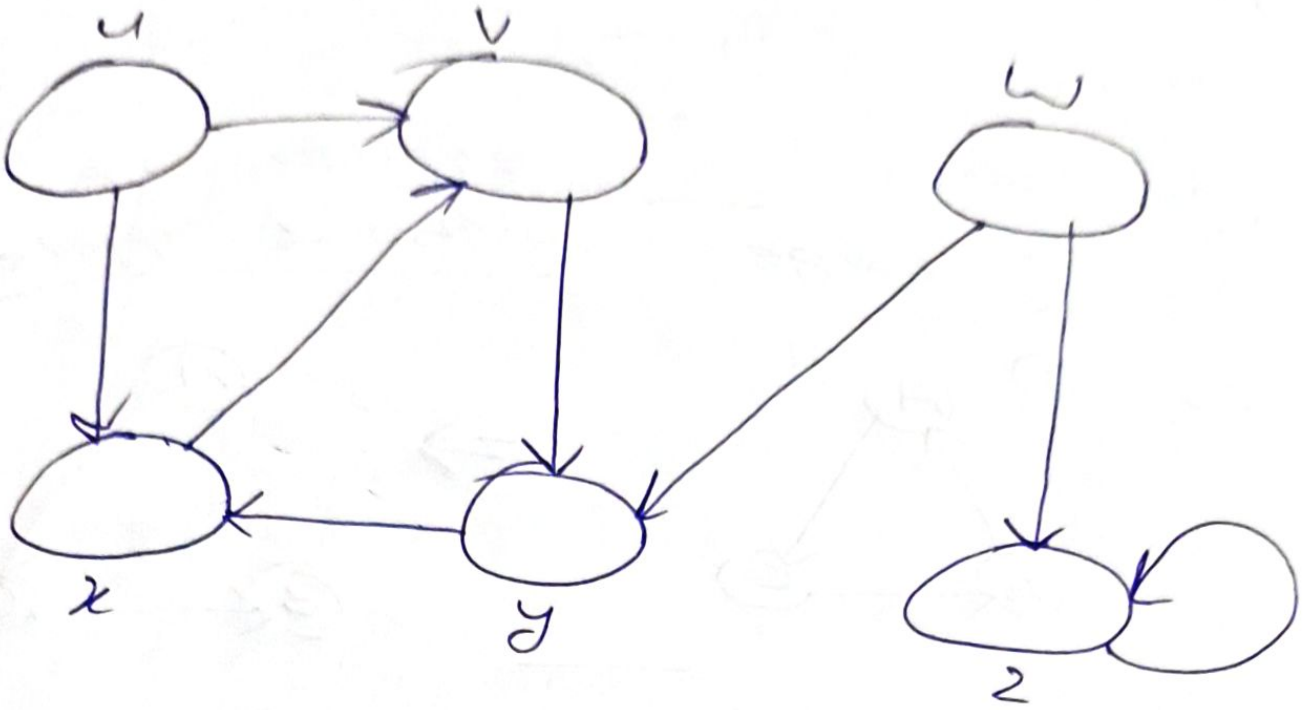
④ Do if color[v] ← white

⑤ Then $\pi[v] \leftarrow u$

⑥ DFS_VISIT[v]

⑦ color[u] ← black

⑧ f[u] ← time ← time + 1




```
graph = {
  'u' : ['v', 'x'],
  'v' : ['u'],
  'y' : ['x'],
  'x' : ['v'],
  'w' : ['y', 'z'],
  'z' : ['z']
}
```

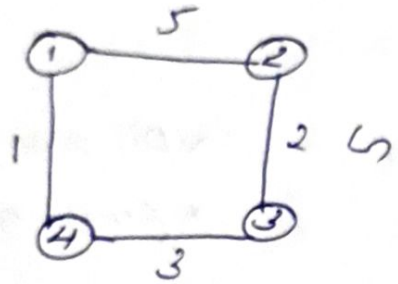
v Discovery time
* finishing time

Minimum Spanning Tree (MST)

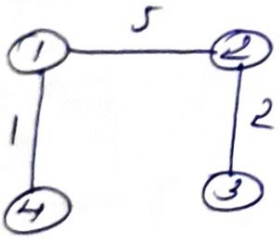
- It have 'n' vertices and 'n-1' edges without 'cycle'

Ex :- (1)

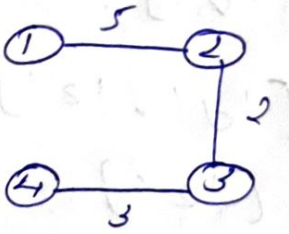
(2)



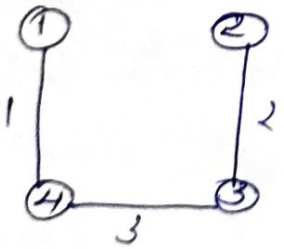
Spanning trees of G are :-



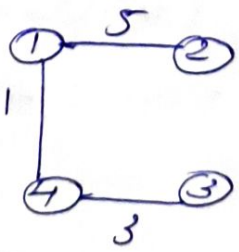
cost = 8



cost = 10

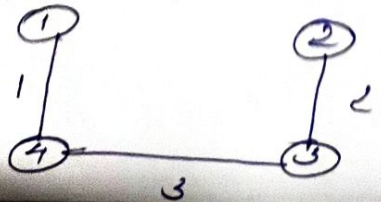


Cost = 6



Cost = 9

Here, Minimum cost spanning tree :-



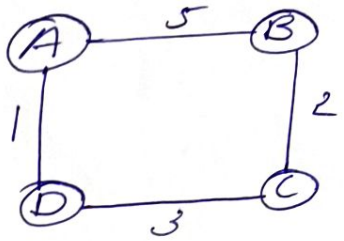
7 Minimum Spanning Tree (MST)

A spanning tree of a connected graph is a connected acyclic subgraph which contains all the vertices of the graph.

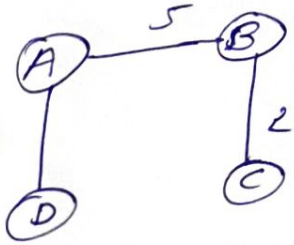
. A spanning tree have 'n' vertices, minimum 'n-1' edges without any cycle.

. MST is a spanning tree with minimum weight, where weight is defined as sum of weights of edges used in the construction of spanning tree.

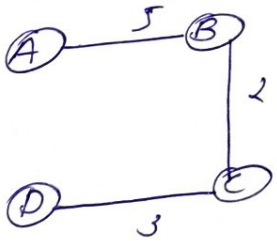
Eg:-



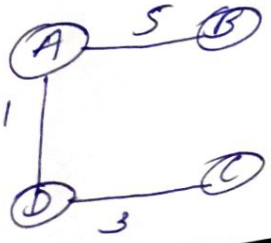
Create spanning tree from this graph. Find minimum cost.



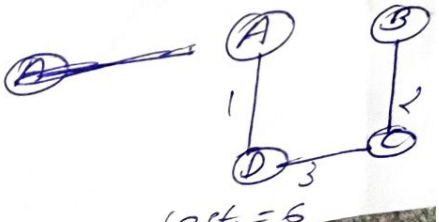
Cost = 8



Cost = 10



Cost = 9



Cost = 6

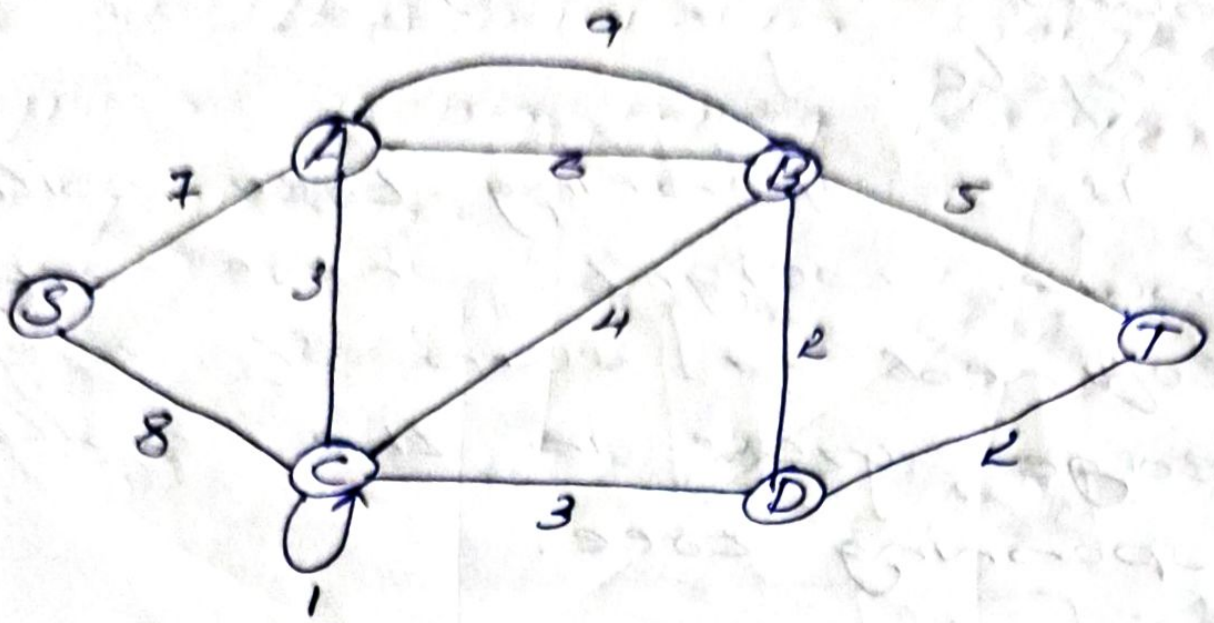
. MST can be constructed from graph using two algorithms :-

① Kruskal's algorithm

② Prim's algorithm

11/2025
Monday

> Kruskal's algorithm



> Algorithm

MST_KRUSKAL (G, w)

1. $A = \phi$
2. For each vertex $V \in G(V, E)$
3. MAKE_SET (V)
4. Sort edge of G(V,E) into non-decreasing order by weight w.
5. For each edge $(u, v) \in V(G)$ taken in non-decreasing order by weight
6. If FIND_SET (u) ≠ FIND_SET (v)
7. $A = A \cup \{(u, v)\}$
8. UNION (u, v)
9. Return A

Steps

① Remove loops and parallel edges
② Arrange edges in increasing order of weight.
③ Arrange edges in order except cycle.

. MST_KRUSKAL find a minimum spanning tree for a connected weighted graph.
. It finds a safe edge to add to the growing forest. → no cycle
. Two operation in KRUSKAL :-

① MAKE_SET
② FIND_SET

> . MAKE_SET operation determines each vertex in non-decreasing order.
> . FIND_SET operation determine whether two vertices u and v belongs to same tree or not.
. To combine trees, KRUSKAL's algorithm calls a procedure called 'UNION'.

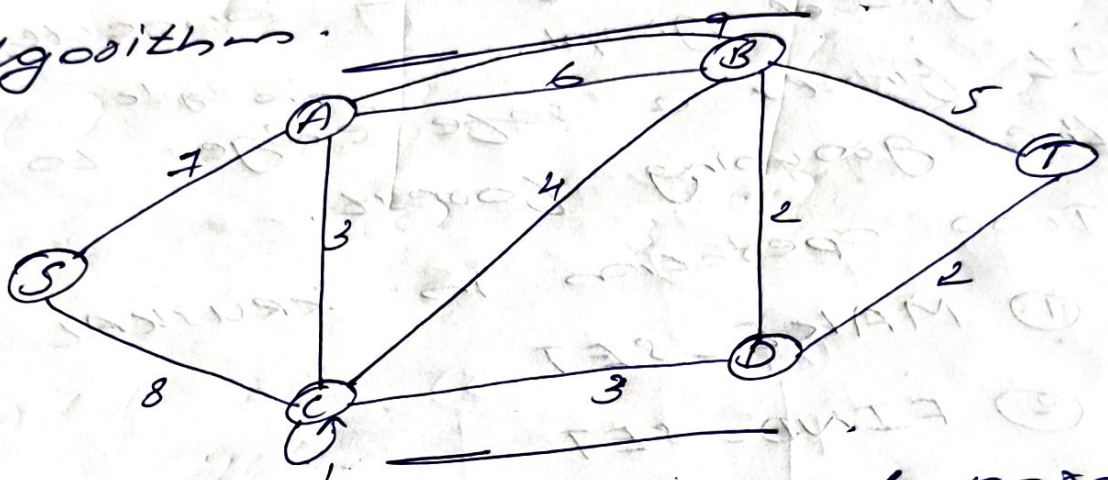
Steps

① Remove loops and parallel edges

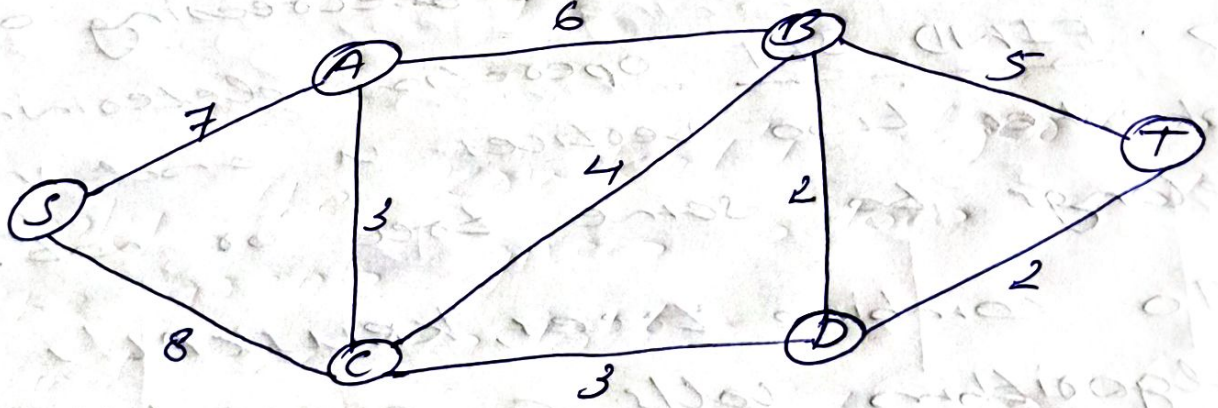
② Sort the edges in increasing order of weight.

Problem

① Find the minimum (MST) weight for the given graph using Kruskal's algorithm.



Ans:- **Step 1:-** Remove loops and parallel edges

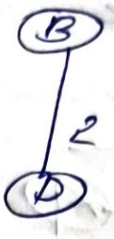


Step 2:- Arrange edges in ascending order.

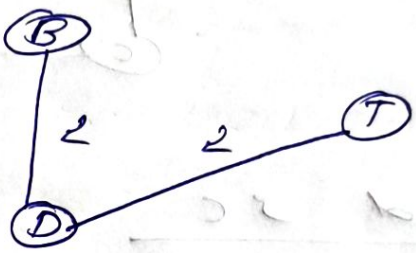
Edge	BD	DT	CD	AC	BC	BT	AB	AS	SC
weight	2	2	3	3	4	5	6	7	8

Step3 :- Construct MST

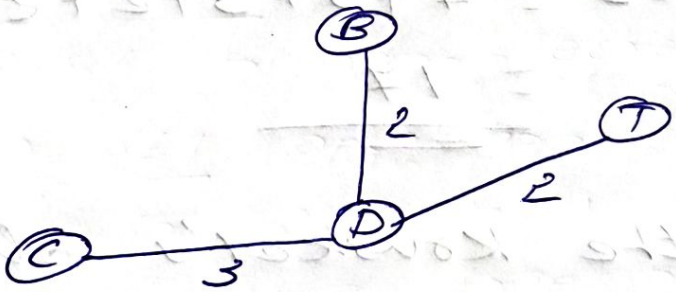
. Add BD



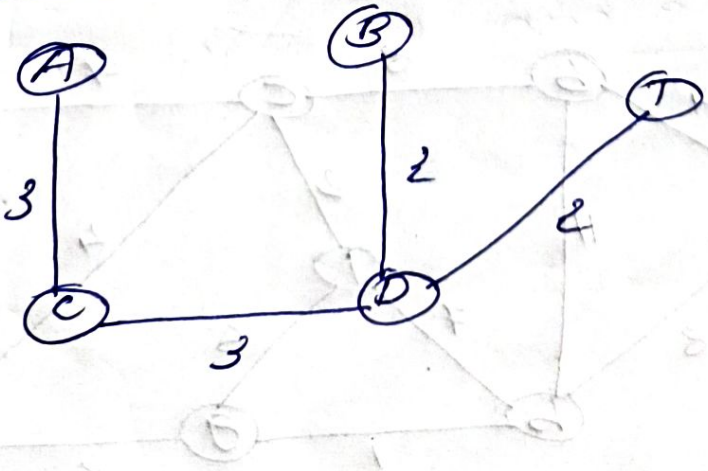
. Add DT



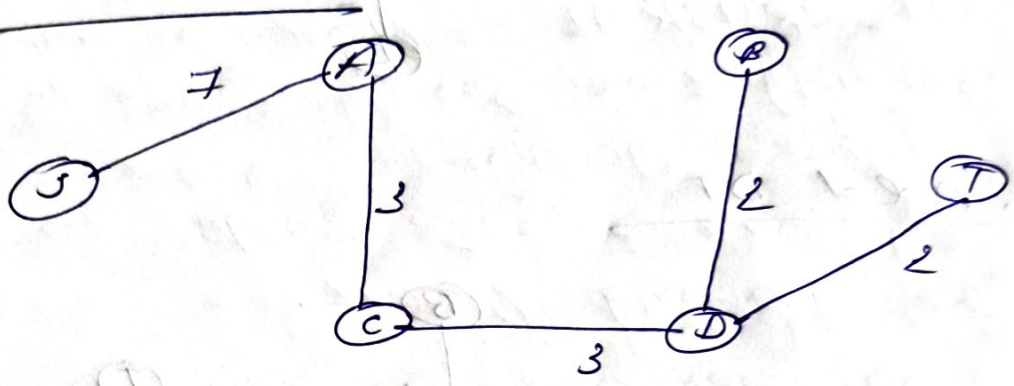
. Add CD



. Add AC



- Add BC
  Cycle
- Add BT
  Cycle
- Add AB
  cycle
- Add AS

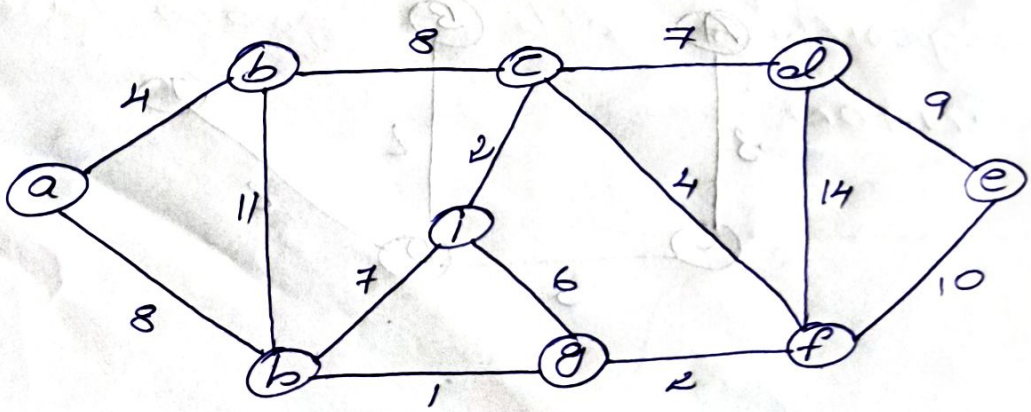


- Add SC
  cycle.

∴ Cost = 7 + 3 + 3 + 2 + 2

= 17

(2) Apply the Kruskal's algorithm for the graph given below and find the weight:-



step1 :- Remove loops & parallel edges

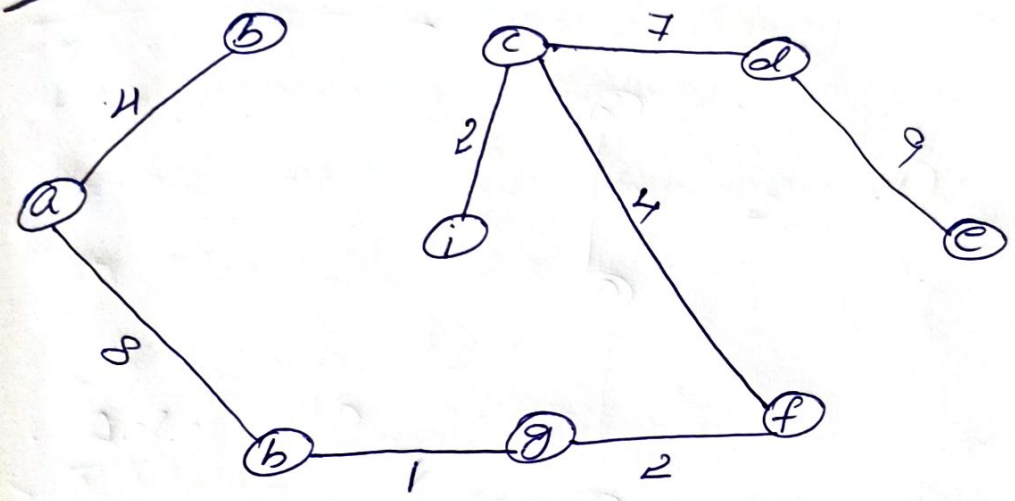
NO loops & parallel edges

step2 :-

Edge	bg	ci	gf	ab	cf	gi	cd	hi	ah	bc	de
weight	1	2	2	4	4	6	7	7	8	8	9

ef	Bh	df
10	11	14

step3 :-

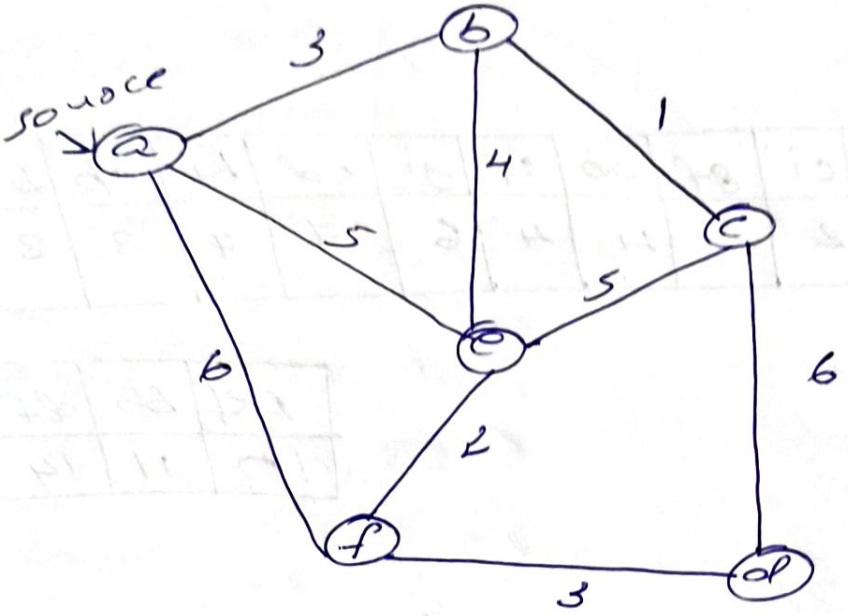


Weight = 4 + 8 + 1 + 2 + 2 + 4 + 7 + 9

= 37

18/11/2025
Tuesday.

Prim's Algorithm



- In this graph, $V = \{a, b, c, d, e, f\}$
- If a graph having 6 vertex, then 5 iterations are mandatory.
- Initially,

$$E_T = \{\} \quad \text{and} \quad V_T = \{\}$$

- Initialization :-

$$V_T = \{\}, \quad E_T = \{\}$$

- Iteration 1 :-

$$V_T = \{a\}$$

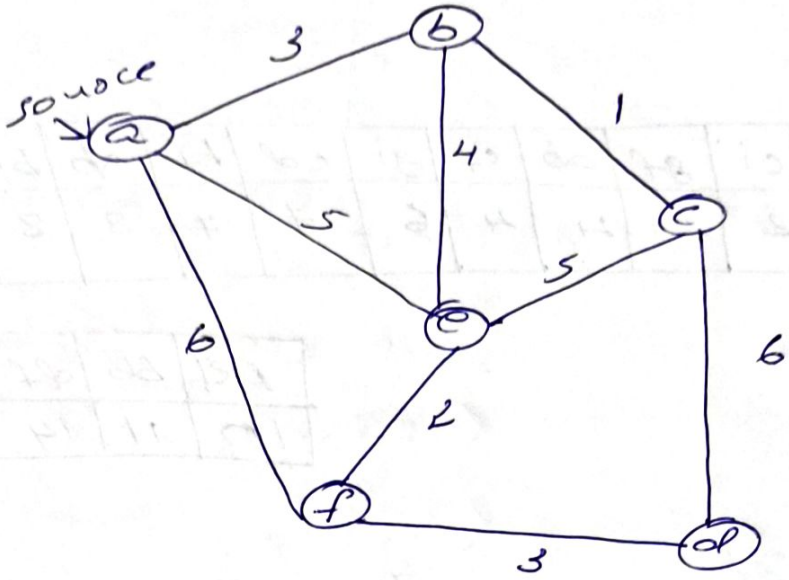
$$V - V_T = \{b, c, d, e, f\}$$

Edges connected to a are :-

Edges	(a,b)	(a,e)	(a,f)
cost	3	5	6

18/11/2025
Tuesday.

Prim's Algorithm



- In this graph, $V = \{a, b, c, d, e, f\}$
- If a graph having 6 vertex, then 5 iterations are mandatory.
- Initially,

$$E_T = \{\} \quad \text{and} \quad V_T = \{\}$$

- Initialization :-

$$V_T = \{\} \quad , \quad E_T = \{\}$$

- Iteration 1 :-

$$V_T = \{a\}$$

$$V - V_T = \{b, c, d, e, f\}$$

Edges connected to a are :-

Edges	(a,b)	(a,e)	(a,f)
cost	3	5	6

So, minimum edge = (a, b)

$\therefore E_T = \{(a,b)\}$

Iteration 2 :-

$V_T = \{a, b\}$

$V - V_T = \{c, d, e, f\}$

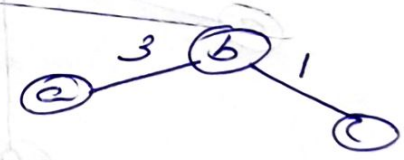
Edges neighbouring to a and b :-

Edges	(a,e)	(a,f)	(b,e)	(b,c)
cost	5	6	4	1

So, minimum edge = (b, c)

$\therefore E_T = \{(a,b), (b,c)\}$

Iteration 3



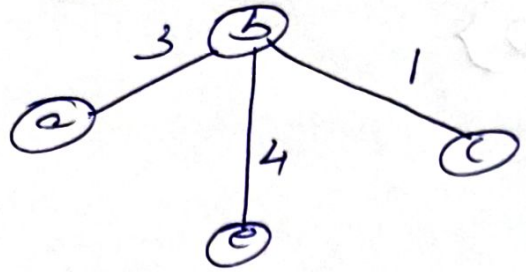
$V_T = \{a, b, c\}$

$V - V_T = \{d, e, f\}$

Edges neighbouring to e, b & c :-

Edges	(a,e)	(a,f)	(b,e)	(c,e)	(,d)
cost	5	6	4	5	6

So, minimum edge = (b, e)



$\therefore E_T = \{(a,b), (b,e), (b,c)\}$

· Iteration 4

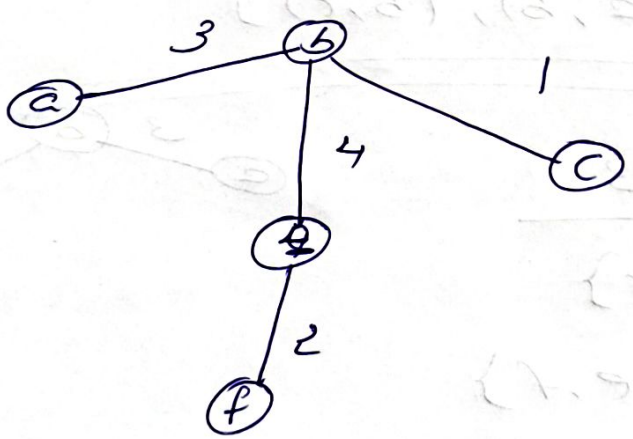
$V_T = \{a, b, c, e\}$

$V - V_T = \{d, f\}$

Edges neighbouring are:-

Edges	(a,e)	(a,f)	(c,d)	(c,e)	(e,f)
Cost	5	6	6	5	2

Minimum edge = (e,f)



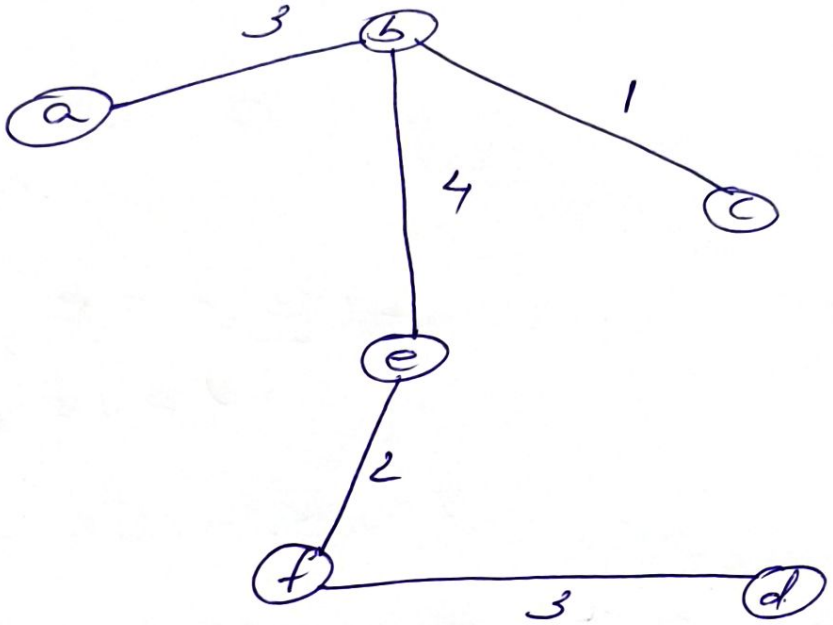
· Iteration 5

$V_T = \{a, b, c, e, f\}$

$V - V_T = \{d\}$

Edges	(a,e)	(a,f)	(c,d)	(c,e)	(e,f)	(f,d)
Cost	5	6	6	5	2	3

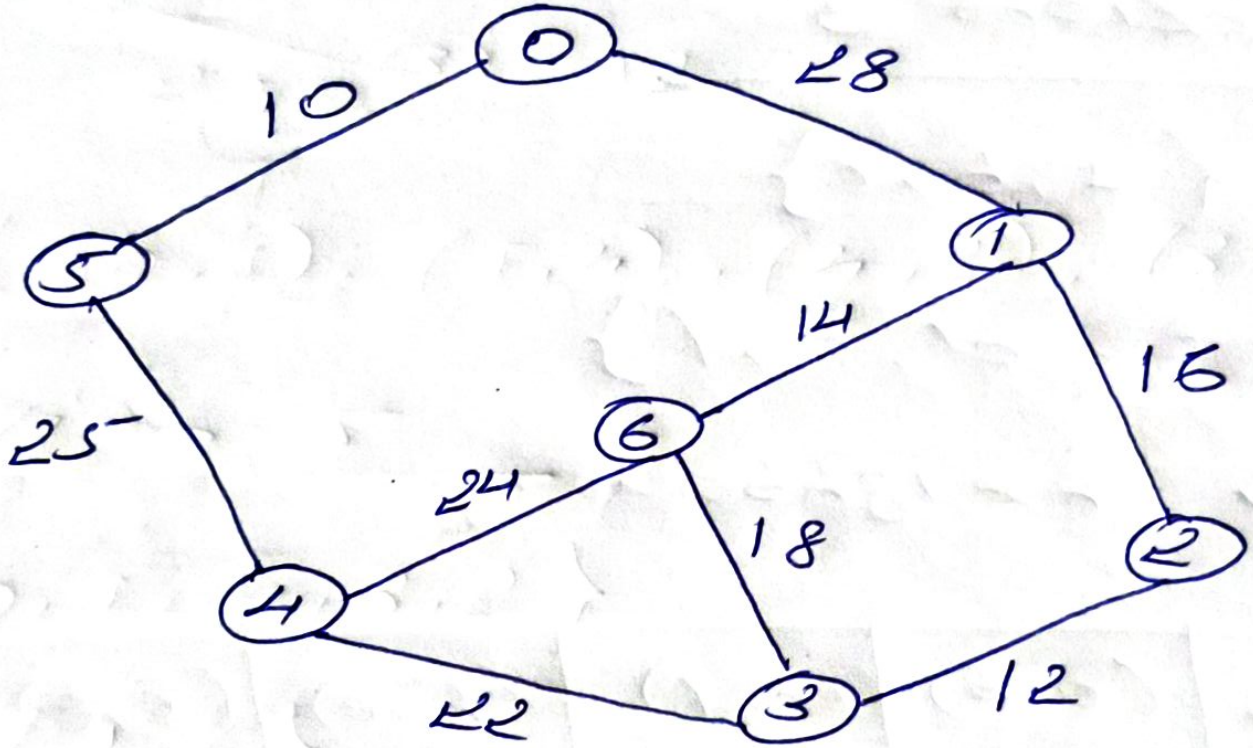
Minimum edge = (f, d)



∴ cost = 3 + 1 + 4 + 2 + 3 = 13

If n vertices, n-1 iterations

2) Apply Prim's algorithm in the follo graph:-



H):- $V = \{0, 1, 2, 3, 4, 5, 6\}$

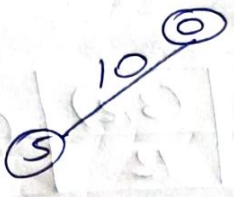
. Iteration 1 :-

$V_T = \{0\}$

$V - V_T = \{1, 2, 3, 4, 5, 6\}$

Edges	(0,5)	(0,1)
Cost	10	28

Minimum edge = (0,5)



$E_T = \{(0,5)\}$

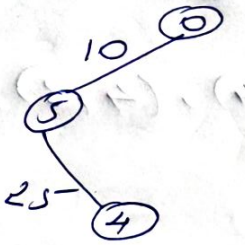
. Iteration 2 :-

$V_T = \{0, 5\}$

$V - V_T = \{1, 2, 3, 4, 6\}$

Edges	(0,1)	(5,4)
Cost	28	25

Minimum edge = (5,4)



$E_T = \{(0,5), (5,4)\}$

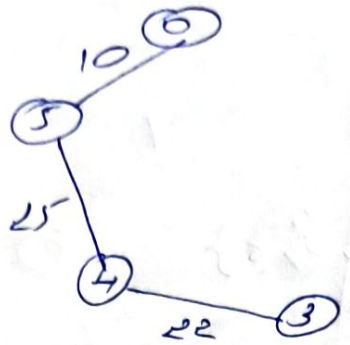
. Iteration 3 :-

$V_T = \{0, 5, 4\}$

$V - V_T = \{1, 2, 3, 6\}$

Edges	(0,1)	(4,6)	(4,3)
Cost	28	24	22

Minimum edge = (4,3)



$\therefore E_T = \{(0,1), (5,4), (4,3)\}$

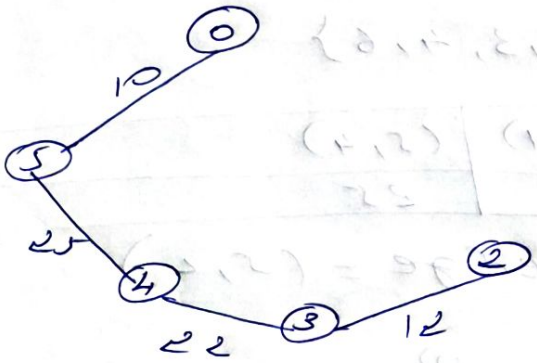
- Iteration 4

$V = \{0, 5, 4, 3\}$

$V - V_T = \{1, 2, 6\}$

Edges	(0,1)	(4,6)	(3,2)	(3,6)
Cost	28	24	12	18

Minimum edge = (3,2)



$E_T = \{(0,5), (5,4), (4,3), (3,2)\}$

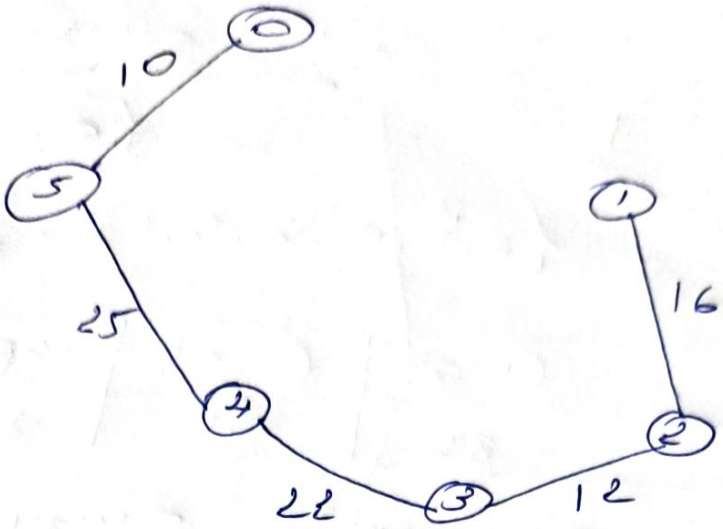
- Iteration 5

$V = \{0, 5, 4, 3, 2\}$

$V - V_T = \{1, 6\}$

Edges	(0,1)	(4,6)	(3,6)	(2,1)
Cost	28	24	18	16

Minimum edge = (2,1)



$E_T = \{(0,5), (5,4), (4,3), (3,2), (2,1)\}$

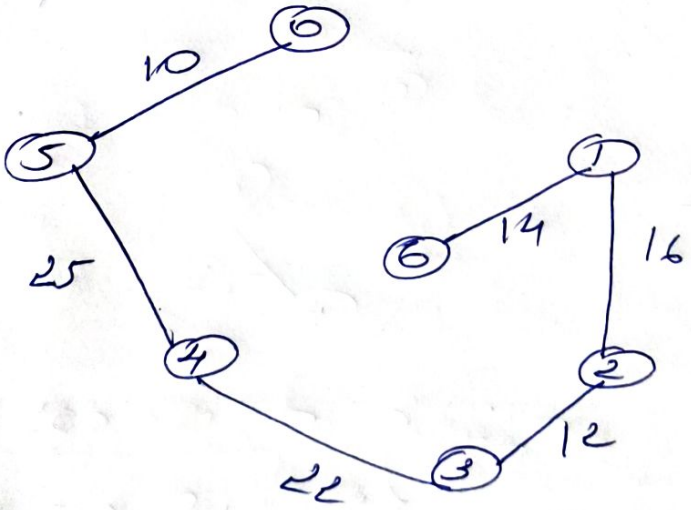
Iteration 6

$V = \{0, 5, 4, 3, 2, 1\}$

$V - V_T = \{6\}$

Edges	(0,1)	(4,6)	(3,6)	(1,6)	~~(4,~~
cost	28	24	18	14	

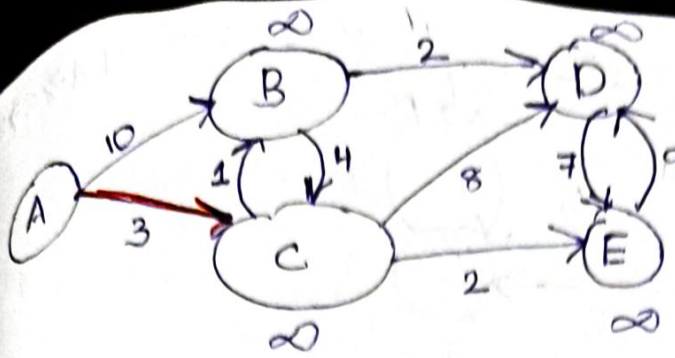
Minimum edge = (1,6)



$\therefore E_T = \{(0,5), (5,4), (4,3), (3,2), (2,1), (1,6)\}$

Hence, Minimum cost = 99

(1)



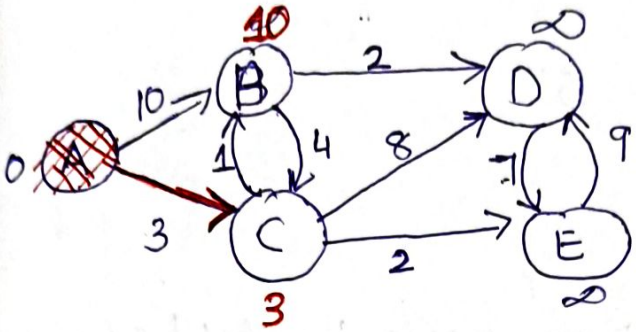
Q: A B C D E
0 ∞ ∞ ∞ ∞

S = {A}

↳ entrat min

S B →

(2)

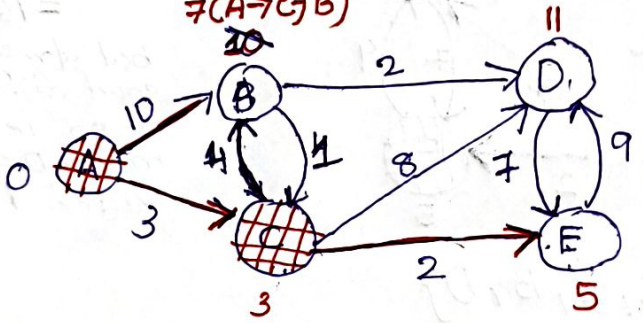


S = {A}

Q: A B C D E
0 ∞ ∞ ∞ ∞
10 3 - -

↳ entrat min → (C). so goto C.

(3)



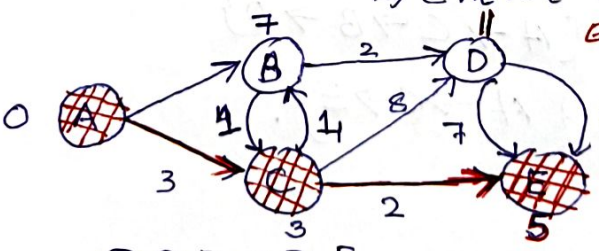
S = {A, C}.

Q: A B C D E
0 ∞ ∞ ∞ ∞
10 3 - -
7 11 5

↳ entrat. min

E→D = 14, D = 11 so 11.

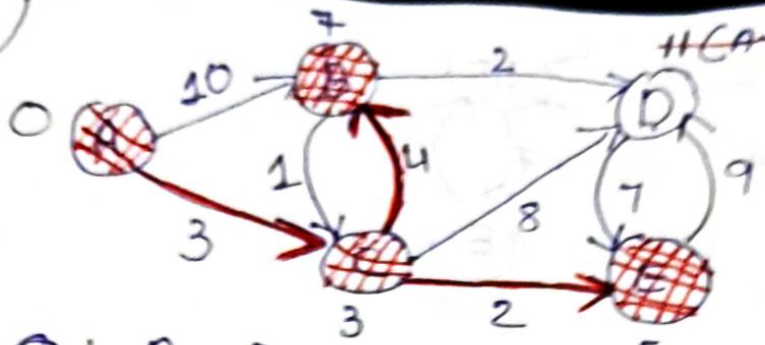
(4)



S = {A, C, E}.

Q: A B C D E
0 ∞ ∞ ∞ ∞
10 3 - -
7 11 5
7 11 → entrat min = B.

(5)



~~11(A→C→E→D)~~ 9(A→C→B→D)

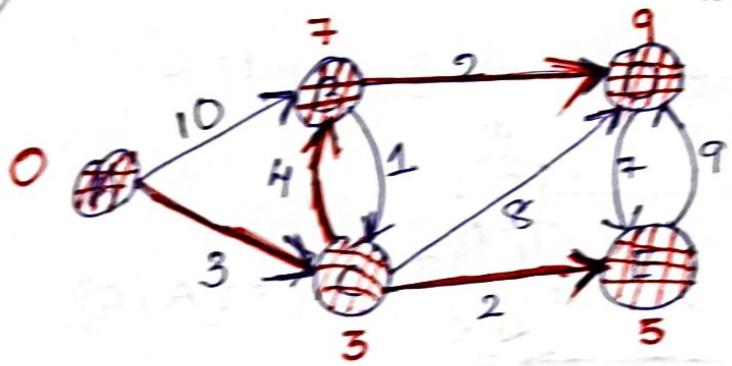
Q:	A	B	C	D	E
A	0	∞	∞	∞	∞
A		10	3		
A·C		7	11		5
A·C·E		7	11		
A·C·B			9		

↳ extract min here only: 9 only all visited 'D' → check all with all vertext connected to D and find distance

D-E D+edge to E
distance = 9+7
= 16

but already the short dist of E is 5 so leave mark D as visited add to S{}

(6) S = {A, C, E, B}



S = {A, C, E, B, D}

shortest distance of A to {A, B, C, D, E}

A → A : 0

A → B : 7 {A → C → B}

A → C : 3 {A → C}

A → D : 9 {A → C → B → D}

A → E : 5 {A → C → E}.